

PROJECT REPORT

HOMEWORK 7

Written and Prepared by:

Roxanne Lutz

University of New Mexico

December 2, 2025

PROBLEM 1

ORIGINAL PROBLEM STATEMENT

“Perform Taylor Series expansions to derive formulas for the following derivatives and determine the order of the approximation:

- a) $u'(x_i)$ using two points x_i, x_{i+1}*
- b) $u'(x_i)$ using two points x_{i-1}, x_i*
- c) $u'(x_i)$ using two points x_{i-1}, x_{i+1}*
- d) $u'(x_i)$ using four points $x_i, x_{i+1}, x_{i+2}, x_{i+3}$ ”*

Below is the handwork done to solve these problems.

Prob 1) Derivative Approximations. $\rightarrow$ Taylor expansion & determine order of approx.

(a) $u'(x_i)$ using x_i, x_{i+1} .

Let $h = |x_{i+1} - x_i| \cdot h$.

So, $u'(x_i) = \lim_{h \rightarrow 0} \frac{u(x_i + \underbrace{x_{i+1} - x_i}_h) - u(x_i)}{(x_{i+1} - x_i)} = \lim_{h \rightarrow 0} \frac{u(x_{i+1}) - u(x_i)}{h}$

$u(x_{i+1}) = u(x_i + h) = u(x_i) + h u'(x_i) + \frac{1}{2} h^2 u''(\xi)$ where $x_i \leq \xi \leq x_{i+1}$.

$u(x_{i+1}) - u(x_i) = h u'(x_i) + O(h^2)$.

$$\boxed{\frac{u(x_{i+1}) - u(x_i)}{h} = u'(x_i) + O(h)}$$

The approximation is first-order ($O(h)$ error term).

This is significant since, w/ the two points, the approximation error increases when the two points are further away from each other (larger $h \rightarrow$ more error) & if the points are closer, the approximation will be more accurate.

(b) $u'(x_i)$ w/ x_{i-1}, x_i .

$h = x_i - x_{i-1} \rightarrow x_i - h = x_{i-1}$.

$u(x_{i-1}) = u(x_i - h) = u(x_i) - h u'(x_i) + \frac{1}{2} h^2 u''(\xi)$, $-x_i \leq \xi \leq x_{i-1}$.

$u(x_{i-1}) - u(x_i) = -h u'(x_i) + O(h^2)$.

$$\boxed{\frac{u(x_i) - u(x_{i-1})}{h} = u'(x_i) + O(h)}$$

This approximation is also first-order accurate (still $O(h)$).
 This has the same significance as part A.

(c) $u'(x_i)$ w/ x_{i-1}, x_{i+1}

$$x_i \leq \xi \leq x_i + h$$

$$u(x_{i+1}) = u(x_i + h) = u(x_i) + h u'(x_i) + \frac{1}{2} h^2 u''(x_i) + \frac{1}{6} h^3 u'''(\xi)$$

$$u(x_{i-1}) = u(x_i - h) = u(x_i) - h u'(x_i) + \frac{1}{2} h^2 u''(x_i) - \frac{1}{6} h^3 u'''(\xi)$$

$$u(x_{i+1}) - u(x_{i-1}) = 2h u'(x_i) + \frac{1}{3} h^3 u'''(\xi)$$

$$u(x_{i+1}) - u(x_{i-1}) = 2h u'(x_i) + O(h^3)$$

$$2h$$

$$\frac{u(x_{i+1}) - u(x_{i-1}))}{2h} = u'(x_i) + O(h^2)$$

This approximation is second order accurate.

Using the 2 points w/ this scheme means an even worse accuracy when the h step is large. using 2 points around x_i .

(d) $u'(x_i)$ w/ 4 points: $x_i, x_{i+1}, x_{i+2}, x_{i-1}$.

$$u(x_{i+1}) = u(x_i + h)$$

$$u(x_{i+2}) = u(x_i + 2h)$$

$$u(x_{i+3}) = u(x_i + 3h)$$

$$u(x_{i+1}) = u(x_i) + hu'(x_i) + \frac{1}{2}h^2 u''(x_i) + \frac{1}{6}h^3 u'''(x_i) + O(h^4)$$

$$u(x_{i+2}) = u(x_i) + 2hu'(x_i) + \frac{1}{2}4h^2 u''(x_i) + \frac{1}{6}8h^3 u'''(x_i) + O(h^4)$$

$$u(x_{i+3}) = u(x_i) + 3hu'(x_i) + \frac{1}{2}9h^2 u''(x_i) + \frac{1}{6}27h^3 u'''(x_i) + O(h^4)$$

$$u(x_i) = u(x_i) + 0 + 0 + 0$$

Goal:

$$u(x_i) + u(x_{i+1}) + u(x_{i+2}) + u(x_{i+3}) \approx u'(x_i) \cdot (h^2) \cdot (h^2) = O(h^2)$$

or better.

$$a_0 u(x_i) + a_1 u(x_{i+1}) + a_2 u(x_{i+2}) + a_3 u(x_{i+3}) \approx u'(x_i)$$

$$a_0 u(x_i) = a_0 u(x_i) + (0)hu'(x_i) + (0)\frac{1}{2}h^2 u''(x_i) + (0)\frac{1}{6}h^3 u'''(x_i)$$

$$a_1 u(x_{i+1}) = a_1 u(x_i) + (2a_1)hu'(x_i) + (1a_1)\frac{1}{2}h^2 u''(x_i) + (1a_1)\frac{1}{6}h^3 u'''(x_i)$$

$$a_2 u(x_{i+2}) = a_2 u(x_i) + (2a_2)hu'(x_i) + (4a_2)\frac{1}{2}h^2 u''(x_i) + (8a_2)\frac{1}{6}h^3 u'''(x_i)$$

$$a_3 u(x_{i+3}) = a_3 u(x_i) + (3a_3)hu'(x_i) + (9a_3)\frac{1}{2}h^2 u''(x_i) + (27a_3)\frac{1}{6}h^3 u'''(x_i)$$

What we want to cancel or produce (solve $u'(x_i)$).

$$a_0 + a_1 + a_2 + a_3 = 0$$

$$a_1 + 2a_2 + 3a_3 = \frac{1}{h}$$

$$a_1 + 4a_2 + 9a_3 = 0$$

$$a_1 + 8a_2 + 27a_3 = 0$$

we'll say 1 for ease of math.
in matrix
Solve the system of equations

$$\left[\begin{array}{cccc|c} 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 2 & 3 & 1 \\ 0 & 1 & 4 & 9 & 0 \\ 0 & 1 & 8 & 27 & 0 \end{array} \right] \xrightarrow[R4-R3]{R1-R2} \left[\begin{array}{cccc|c} 1 & 0 & -1 & -2 & -1 \\ 0 & 1 & 2 & 3 & 1 \\ 0 & 1 & 4 & 9 & 0 \\ 0 & 0 & 4 & 18 & 0 \end{array} \right] \xrightarrow[R3-R2]{R3-R2} \left[\begin{array}{cccc|c} 1 & 0 & -1 & -2 & -1 \\ 0 & 1 & 2 & 3 & 1 \\ 0 & 0 & 2 & 6 & -1 \\ 0 & 0 & 4 & 18 & 0 \end{array} \right]$$

$$\xrightarrow{R4-2R3} \left[\begin{array}{cccc|c} 1 & 0 & -1 & -2 & -1 \\ 0 & 1 & 2 & 3 & 1 \\ 0 & 0 & 2 & 6 & -1 \\ 0 & 0 & 0 & 6 & 2 \end{array} \right] \xrightarrow[4R4]{1/2 R3} \left[\begin{array}{cccc|c} 1 & 0 & -1 & -2 & -1 \\ 0 & 1 & 2 & 3 & 1 \\ 0 & 0 & 1 & 3 & -1/2 \\ 0 & 0 & 0 & 1 & 1/3 \end{array} \right] \xrightarrow[3R4]{R2-} \left[\begin{array}{cccc|c} 1 & 0 & -1 & -2 & -1 \\ 0 & 1 & 2 & 0 & 0 \\ 0 & 0 & 1 & 0 & -3/2 \\ 0 & 0 & 0 & 1 & 1/3 \end{array} \right] \xrightarrow[3R4]{R3+} \left[\begin{array}{cccc|c} 1 & 0 & -1 & -2 & -1 \\ 0 & 1 & 2 & 0 & 0 \\ 0 & 0 & 1 & 0 & -3/2 \\ 0 & 0 & 0 & 1 & 1/3 \end{array} \right] \xrightarrow[3R4]{R2-} \left[\begin{array}{cccc|c} 1 & 0 & -1 & -2 & -1 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & -3/2 \\ 0 & 0 & 0 & 1 & 1/3 \end{array} \right] \xrightarrow[3R4]{R1+} \left[\begin{array}{cccc|c} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & -3/2 \\ 0 & 0 & 0 & 1 & 1/3 \end{array} \right]$$

$$R2 - 2R3 \rightarrow \begin{bmatrix} 1 & 0 & -1 & -2 & -1 \\ 0 & 1 & 0 & 0 & 3 \\ 0 & 0 & 1 & 0 & -3/2 \\ 0 & 0 & 0 & 1 & 1/3 \end{bmatrix} \xrightarrow{\substack{R1 + R3, \\ R1 + 2R4}} \begin{bmatrix} 1 & 0 & 0 & 0 & -11/6 \\ 0 & 1 & 0 & 0 & 3 \\ 0 & 0 & 1 & 0 & -3/2 \\ 0 & 0 & 0 & 1 & 1/3 \end{bmatrix} \begin{matrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{matrix}$$

$-\frac{11}{6} - 3/2 + \frac{2}{3}$
 $-\frac{5}{6} + \frac{2}{3}$
 $-\frac{15}{6} + \frac{4}{6}$
 $= -\frac{11}{6}$

SO:

$$-\frac{11}{6}u(x_i) + 3u(x_{i+1}) - \frac{3}{2}u(x_{i+2}) + \frac{1}{3}u(x_{i+3}) \approx h u'(x) + O(h^4)$$

$$\frac{-\frac{11}{6}u(x_i) + 3u(x_{i+1}) - \frac{3}{2}u(x_{i+2}) + \frac{1}{3}u(x_{i+3})}{h} = u'(x) + O(h^3)$$

This approximation is third order ($O(h^3)$ error term).

A small h would make this an excellent approximation technique!

PROBLEM 2

ORIGINAL PROBLEM STATEMENT (*truncated*)

“Use (*) or (**) to obtain a numerical approximation for $u(x)$ for the following problem:

$$u''(x) + 3u'(x) + 2u(x) = 4x^2, u(1) = 1, u(2) = 6$$

a) Use $N = 10$ to plot the approximation, $u(x)$ vs x .

b) Use $N = 40$ to plot the approximation along with the exact solution $u_{\text{exact}}(x)$ ”

For this problem, we will use the following code distributed for use.

```
%
=====
(1) CODE WRITTEN FOR PLOTTING IN sample_CODE_version2.m FILE
%
=====

% Sample Code for Finite Difference method
clear all All
a=1;
b=2;
%number of subintervals
n=10;
h=(b-a)/n;
%unknowns n-2
LBC=1;
RBC=6;
A=zeros(n-1,n-1);
P=3;Q=2;

%main diagonal
for i=1:n-1
    A(i,i)=(-2+h^2*Q);
end
for i=1:n-2
    %lower diag
    A(i+1,i)=(1-h/2*P);
    %upper diagonal
    A(i,i+1)=(1+h/2*P);
```

```

end

%right hand side
x=[a+h:h:b-h];
f=h^2*4*x.^2;
%right hand side contributions with the bcs
f(1)=f(1)-(1-h/2*P)*(LBC);    f(end)=f(end)-(1+h/2*P)*(RBC);
%simply solve Au=f, we take f' since it is to be a column vector
u=A\f';
%pad the solution
x=[a x b];
u=[LBC u' RBC];

%building uexact
C1 = -(exp(3)*(2 + 3*exp(1))) / (exp(1) - 1);
C2 = (2*exp(1) + 3*exp(3)) / (exp(1) - 1);
% Define uexact
uexact = 7 - 6*x + 2*x.^2 + C1*exp(-2*x) + C2*exp(-x);

figure(1)
plot(x,u,'o-',x,uexact,'rx')
title("uapprox VS uexact, n = " + n)
legend('uapprox','exact')
xlabel('x')
ylabel('u')
ax = gca;
ax.FontSize = 18;
%error plots
figure(2)
plot(x,abs(u-uexact),'b-')
title(['Error Plot, n = ' num2str(n)])
xlabel('x')
ax = gca;
ax.FontSize = 18;

% repeat of above but with N = 40 for second part
% Sample Code for Finite Difference method
a=1;
b=2;
%number of subintervals
n=40;

```



```

h=(b-a)/n;
%unknowns n-2
LBC=1;
RBC=6;
A=zeros(n-1,n-1);
P=3;Q=2;

%main diagonal
for i=1:n-1
    A(i,i)=(-2+h^2*Q);
end
for i=1:n-2
    %lower diag
    A(i+1,i)=(1-h/2*P);
    %upper diagonal
    A(i,i+1)=(1+h/2*P);
end

%right hand side
x=[a+h:h:b-h];
f=h^2*4*x.^2;
%right hand side contributions with the bcs
f(1)=f(1)-(1-h/2*P)*(LBC);    f(end)=f(end)-(1+h/2*P)*(RBC);
%simply solve Au=f, we take f' since it is to be a column vector
u=A\f';
%pad the solution
x=[a x b];
u=[LBC u' RBC];

%building uexact
C1 = -(exp(3)*(2 + 3*exp(1))) / (exp(1) - 1);
C2 = (2*exp(1) + 3*exp(3)) / (exp(1) - 1);
% Define uexact
uexact = 7 - 6*x + 2*x.^2 + C1*exp(-2*x) + C2*exp(-x);

figure(3)
plot(x,u,'o-',x,uexact,'rx')
title("uapprox VS uexact, n = " + n)
legend('uapprox','exact')
xlabel('x')
ylabel('u')

```

```

ax = gca;
ax.FontSize = 18;
%error plots
figure(4)
plot(x,abs(u-uexact),'b-')
title(['Error Plot, n = ' num2str(n)])
xlabel('x')
ax = gca;
ax.FontSize = 18;

```

```
%
```

```
=====
```

```
END CODE FOR PROB 2
```

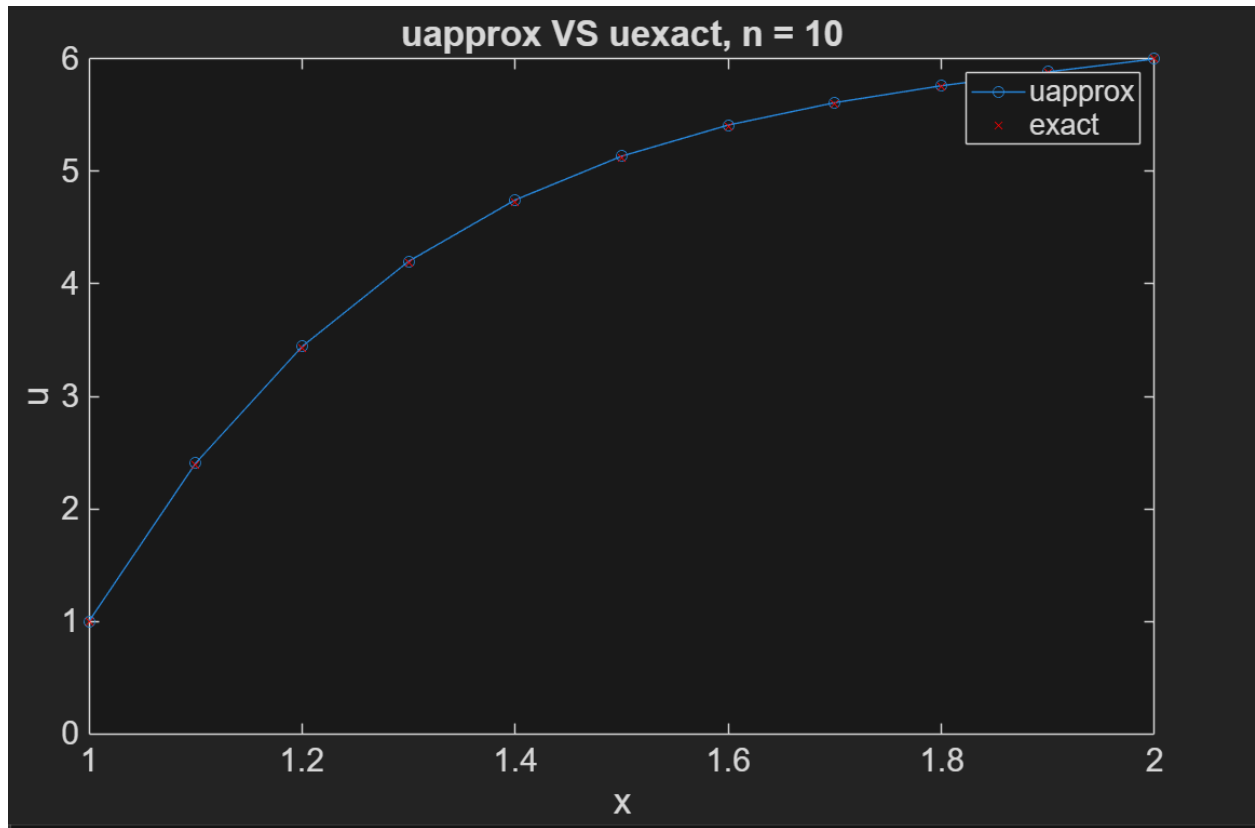
```
%
```

```
=====
```

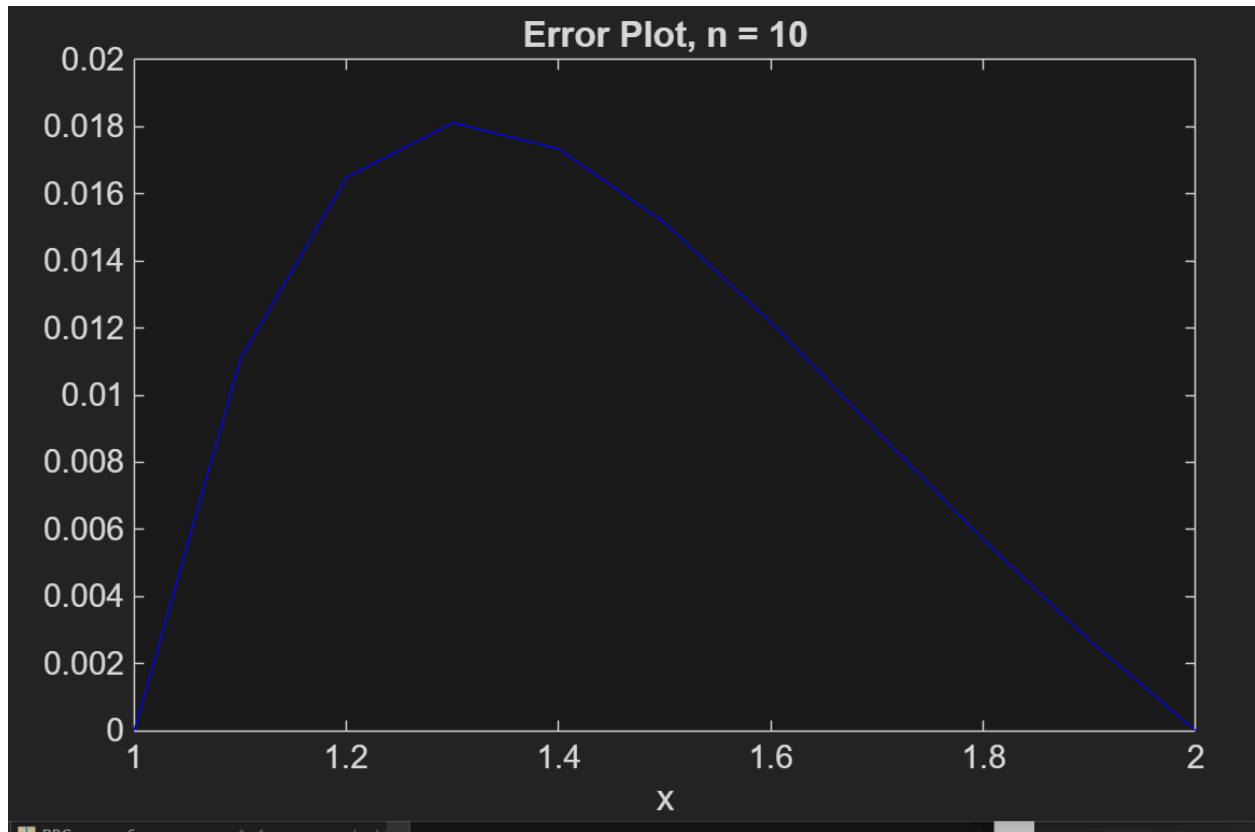
PART A

The scheme that we use for this has two approximations: u' and u'' . A crucial piece of knowledge is that these derivative approximations have error terms depending on the step size h , which correlates directly to the number of sub-intervals in the interval of approximation.

First, we can look at when we have 10 sub-intervals, or $N = 10$ to see how high the error is as a baseline.



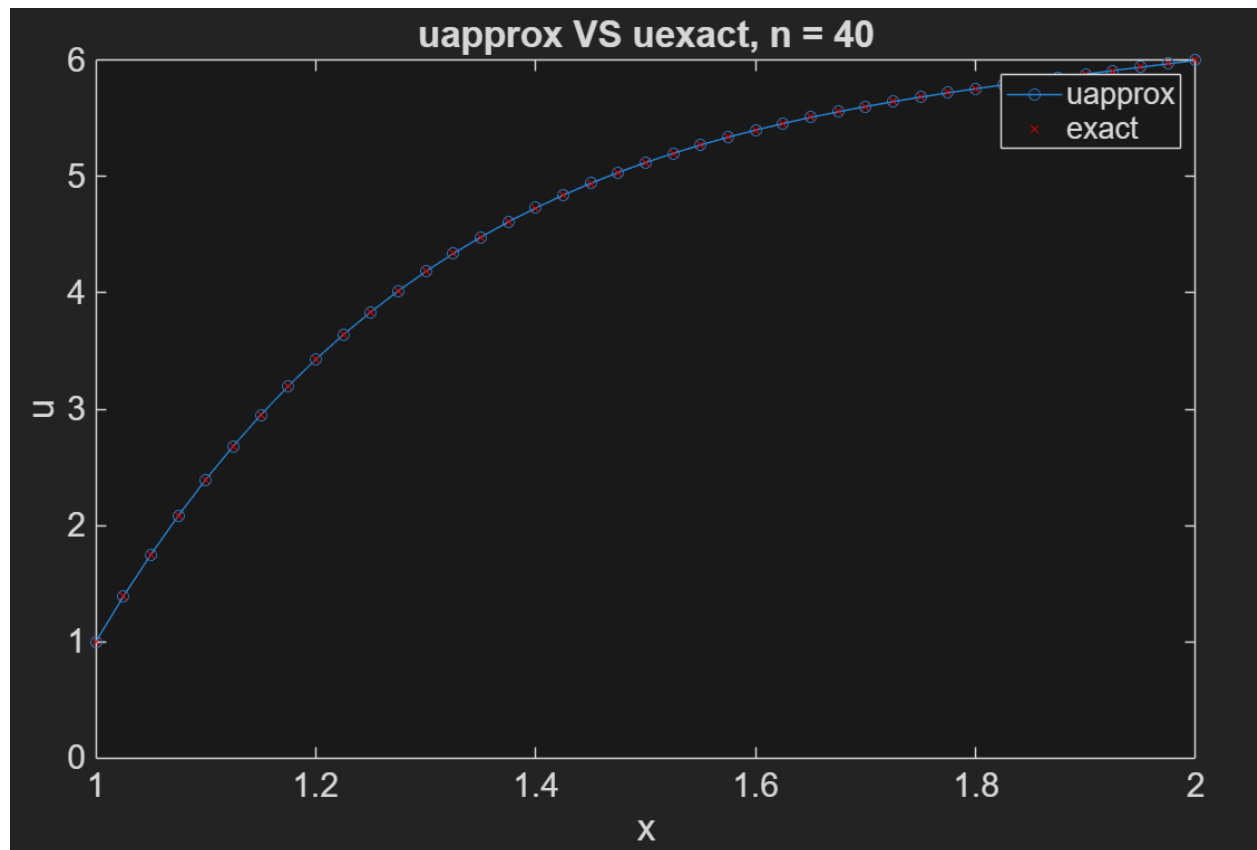
The approximation appears somewhat stiff to the fit, and we can further see that in the error plotting below.



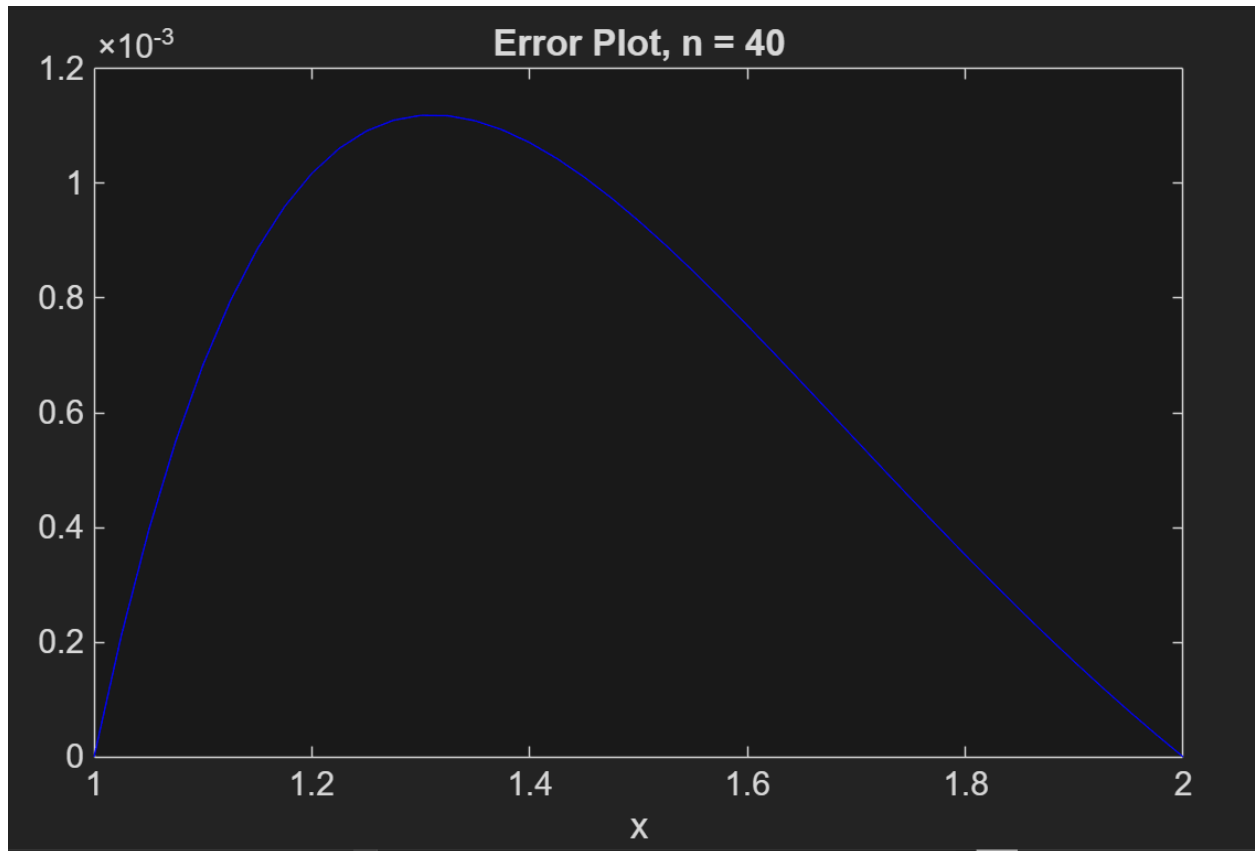
A peak error of about 0.02 is quite high, and likely not very useful in real applications. But, since we know that the approximation error depends on N , we can increase N and it should provide a better approximation.

PART B

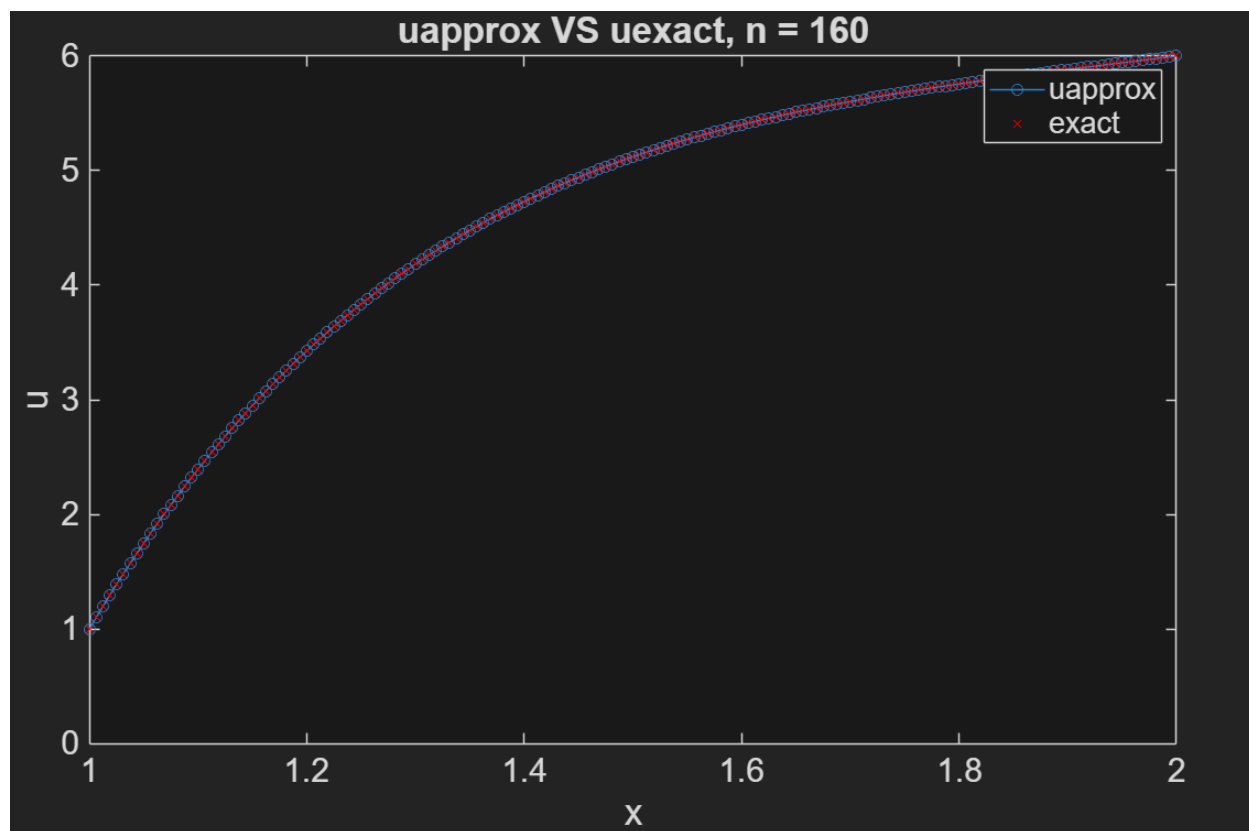
Now, let us examine when $N = 40$.

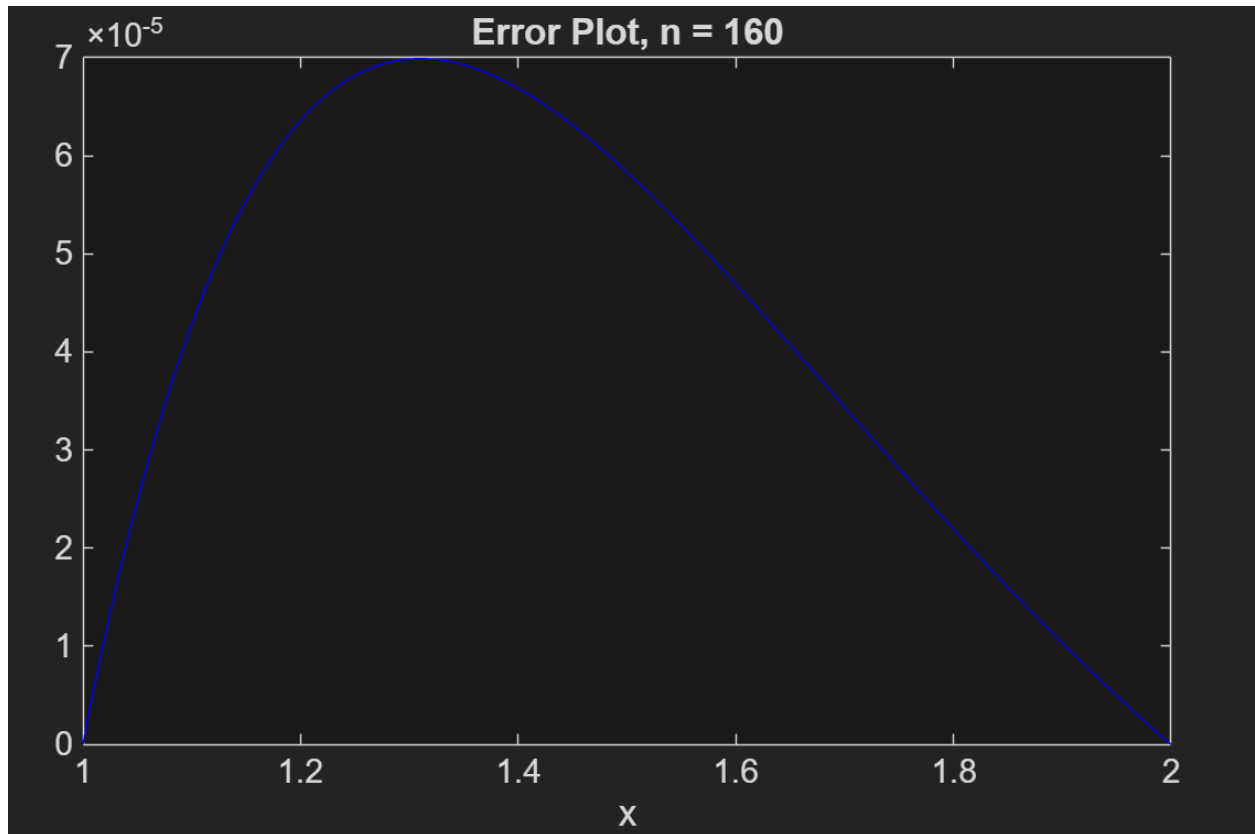


We can already notice a much smoother fit, and the error plotting improves as well.



By simply increasing N to 40 (4 times what it was), we can see the error drop by an order of magnitude to about 0.0012. We can see this one more time behave consistently, increasing N to 160:





The error continues to drop, this time to an even better order of 10^{-5} , making the method sound in approximation given enough sub-intervals in the domain we are using. Further, its consistency makes it a valuable tool, given that the accuracy can be manipulated so readily.

PROBLEM 3

PART A

ORIGINAL PROBLEM STATEMENT

“Determine the number of subintervals N needed to approximate $\int_0, \pi: \sin(x)dx$ with an error not to exceed 10^{-4} using the composite trapezoid rule”

Below is the handwork done to solve these problems.

Prob 3

(a) How many N subintervals needed for error $\leq 10^{-4}$ w/ trapezoid.
 $\int_0^\pi \sin(x) dx.$

Error bound for trapezoid rule:

$$|E| \leq \frac{1}{12} (b-a) h^2 f''(\xi)$$

$$= \left| \frac{1}{12} \frac{(b-a)^3}{N^2} f''(\xi) \right| = \left| \frac{1}{12} \frac{(\pi-0)^3}{N^2} \sin(x) \right|$$

Max val for this on $[0, \pi]$ is 1.

$$= \frac{\pi^3}{12} \left(\frac{1}{N^2} \right) \leq \frac{1}{10^4}$$

$$\Rightarrow \frac{\pi^3 10^4}{12} \leq N^2 \rightarrow \sqrt{\frac{\pi^3 10^4}{12}} \leq N.$$

$$\rightarrow 160.74 \leq N.$$

$\rightarrow N$ should therefore be greater than 160.74, so N should be at least 161 subintervals to achieve a 10^{-4} error.

PART B

ORIGINAL PROBLEM STATEMENT

“Using the result found in a), compute the actual error. Discuss your findings.”

For this problem, we will use the following code distributed for use.

```
%
=====
(1) CODE WRITTEN FOR PLOTTING IN Trap_script.m FILE
%
=====

%Trapezoid script for approximating
%the integral(f(x),{x,a,b}).
clc
clear all
format long
%domain [a,b]
a=0;b=pi;
n=161; %number of points in partition
h=(b-a)/(n);
x=a:h:b;
%little shift from the pseudo-code to MATLAB
sum=0.5*(sin(a)+sin(b));
for i=1:length(x)-2
    x=a+i*h;
    sum=sum+sin(x);
end
sum=sum*h;
fprintf('area approx - our code')
sum

%
=====
END CODE FOR PROB 2
%
=====
```

If we run the above code, we get the following trapezoidal approximation to the integral.

```
area approx - | our code  
sum =  
  
1.999936540083114
```

Now, below is the handwork done to calculate the actual error between this approximate area versus the exact area found by handsolving the integral.

(B) After applying the code, we get an approximation of:

$$\int_0^{\pi} \sin(x) dx = 1.99993654, \quad N=161$$

Real integral:

$$\int_0^{\pi} \sin x dx = -\cos x \Big|_0^{\pi} = -\cos(\pi) + \cos(0) = -1 + 1 = \boxed{2}$$

Actual Error:

$$|E| = |2 - 1.99993654| = 6.346 \cdot 10^{-5} \leq 10^{-4}!$$

→ We end up with a smaller error than the prediction of 10^{-4} ;
which is great.

PART C

ORIGINAL PROBLEM STATEMENT (*truncated*)

“Starting from

$$f'(x) = (f(x + h) - f(x)) / h$$

which gives a first order, $O(h)$, approximation, increase the accuracy by means of applying (1 time only) Richardson Extrapolation to your first derivative formula. Express the form of your formula and include the error term. What order do you get?”

Below is the handwork done to solve these problems.

(C) Richardson Extrapolation.

Taylor Expansion of $f(x+h)$.

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2}f''(x) + O(h^3)$$

$$\frac{f(x+h) - f(x)}{h} = f'(x) + \frac{h}{2}f''(x) + O(h^2)$$

$$f'(x) = \frac{f(x+h) - f(x)}{h} - \frac{h}{2}f''(x) - O(h^2)$$

$$\text{let } \phi(h) = \frac{f(x+h) - f(x)}{h} \approx f'(x)$$

$$\boxed{\phi(h) = f'(x) - a_1 h - a_2 h^2}$$

Consider:

$$2[\phi(h/2) = f'(x) - a_1 \frac{h}{2} - a_2 \frac{h^2}{4}] 2$$

$$2\phi(h/2) = 2f'(x) - a_1 h - a_2 \frac{h^2}{2}$$

$$\boxed{2\phi(h/2) - \phi(h) = f'(x) + O(h^2)}$$

$\hookrightarrow f'(x) \approx 2\phi(h/2) - \phi(h)$ with a second order approximation error ($O(h^2)$).

CONCLUSION

Over the course of this assignment we have seen several derivations and methods for approximating derivatives and integrals, along with applications of such. However, the common thread was the emphasis on being aware of what the error depends upon. With these schemes, there are varying levels of accuracy depending upon changes in sub-interval quantity. The incredible property of these schemes is that they are consistent—that is, a change in the sub-interval number alone will guarantee a great improvement on the approximation. This property can be wielded well by mathematicians and programmers alike to provide highly accurate approximations as needed in scientific realms. This fact alone is a crucial reason to be aware of error sources and to select values wisely with tradeoffs in mind.